Chapter 1: Software and Software Engineering

1.0 The Nature of Software

- Software is intangible: effort, progress, and quality can be difficult to see.
- Reproduction is inexpensive compared with development. Development is labour-intensive and hard to automate completely.
- Easy modification can hide expensive consequences: changes made without understanding the design can introduce defects and complexity.
- Software does not physically wear out. Its structure can deteriorate through poorly planned changes.

1.0.0 Types of software

- Real-time embedded systems monitor or control equipment and must meet timing requirements; safety may be critical.
- Data-processing systems support business operations, where data accuracy and security matter.
- Games, mobile applications, and web applications have different users, operating environments, and quality priorities.

1.1 What is Software Engineering?

Software engineering solves customer problems through systematic development and evolution of large, sufficiently high-quality software systems within cost, time, and other constraints.

- Understand the customer's problem before choosing a solution. Buying or adapting existing software may be appropriate.
- Apply established techniques in an organized way. Much development evolves existing systems.
- Divide work across a team and coordinate the interfaces between parts. One person cannot understand every detail of a large system.
- Estimate resources realistically and ensure benefits justify costs. Extra features can increase complexity without solving the problem.

1.2 The Software Engineering Profession

The term software engineering dates to 1968. An engineering approach combines technical knowledge, disciplined practice, and responsibility for the resulting systems.

1.2.0 Ethics

- Protect the public interest and act responsibly toward clients and employers.
- Aim for high product and maintenance standards, and preserve independent professional judgment.
- Promote ethical management and the integrity of the profession.
- Support colleagues fairly and continue learning throughout a career.

1.3 Software Quality

- Usability: users can learn the system and complete their work effectively.
- Efficiency: the system avoids unnecessary use of processor time, memory, and other resources.
- Reliability: the system performs required functions without failure under stated conditions.
- Maintainability: developers can understand and change the system without excessive effort.
- Reusability: components can serve other projects or applications.

1.3.0 Stakeholders and tradeoffs

- Users value ease of learning and effective task completion; customers value solving a problem at an acceptable total cost.
- Developers value understandable designs and components that are easy to maintain and reuse.
- Development managers balance customer satisfaction with development and maintenance costs.
- Quality goals can conflict. Optimization may make code harder to maintain; a richer interface may consume more resources.
- Set quality objectives before designing. Over-engineering beyond those needs wastes resources.

1.4 Software Engineering Projects

- Corrective maintenance fixes defects.
- Adaptive maintenance responds to environmental changes such as operating systems, databases, or external rules.
- Enhancement adds user-visible capabilities.
- Reengineering or perfective work improves internal structure and maintainability.

- Greenfield development creates a new system. Many projects instead evolve legacy systems.

1.5 Activities Common to Software Projects

1.5.0 Requirements and specification

- Domain analysis develops understanding of the field and problem.
- Gathering obtains information from stakeholders and other sources.
- Analysis organizes and resolves that information. Specification records required behaviour and constraints.

1.5.1 Design

- Systems engineering allocates responsibilities between hardware and software.
- Architecture divides software into subsystems and defines their interactions.
- Detailed design determines subsystem internals, user interfaces, and database structures.

1.5.2 Modelling, implementation, and delivery

- Use-case models describe goals and interactions. Structural models describe organization. Behavioural models describe interactions and changes over time.
- Programming implements the design. Reviews, inspections, and tests assess quality.
- Deployment makes the system available. Process management coordinates work, resources, and progress.
- These activities recur and inform one another as software evolves.